NotesApp Password Audit — Class 5, Lab 2

Pair: HAOJIE SONG (2023905496); ANGRAN YU (2023900899)
Course: Computer Security — EPITA · Kristen Pire
Date: 22 October 2026
Targets: only a dump we generated ourselves from the course's own seed data (`make_dump.py`) and our own NotesApp clone.

Audit summary — hand this in

```
NOTESAPP PASSWORD AUDIT — pair: HAOJIE SONG (2023905496); ANGRAN YU (2023900899)

FINDING 1 — stored passwords (unsalted MD5)
  Cracked:      4/6 in 0.1 ms, public wordlist
  Held out & why: marie (mariepw: a pattern, not a dictionary word) and raj
                (Gv7#pLm2!xQ: long and random); neither is in the public list,
                and cracking only finds passwords that the attacker guesses
  Severity:     High because a leaked table gives 4 of 6 real passwords in under
                a millisecond, on a laptop, with no special tool
  Fix:          store passwords with Argon2 (random salt, slow on purpose) and
                upgrade each old password at its owner's next successful login

FINDING 2 — the fix, measured
  MD5:    3,459,762 guesses/sec   → 298,923,436,800 / day
  Argon2: 10.2 guesses/sec        → 881,280 / day
  Argon2 is ~339,000× slower per guess.
  Verdict: The attacker beats MD5 and gives up on Argon2, because a
           1,000,000-word list takes about 0.3 s against MD5 but about 27 hours
           per account against Argon2 (the random salt makes the attacker
           repeat the work for every account).

STATE: NotesApp now stores passwords with Argon2 (alice migrated on login).  ✅
```

Lab worksheet

Finding 1 — How fast can an attacker read the stored passwords?

Predict first (our answer before the attack — how many of the 6 fall, how long, will all of them fall?): We guess that 4–5 of the 6 may be cracked, and that simple passwords may come out in a few seconds. But not necessarily all of them: if a password is long and random, it may take a very long time to guess. This is only our guess before the attack.

How we ran it. `crack/make_dump.py` and `crack/crack.py` are copied unchanged from the lab page. `crack/wordlist.txt` is the course download `common-passwords.txt`, unchanged: 170 non-empty lines (SHA-256 `3614d52d…bc5e3f`).

```
$ python3 crack/make_dump.py
wrote crack/leaked_users.txt

$ python3 crack/crack.py
cracked 4 / 6 in 0.1 ms
  liwei : 123456
  aisha : password1
  tomk : qwerty123
  sofia : iloveyou
```

Measure — record what we got.

- Accounts cracked: **4 / 6**
- Time taken: **0.1 ms** (one run on our laptop; the time covers the attack loop only)

Each cracked password was hashed again with MD5 and matched its leaked digest.

Debrief — did every account fall? No. marie and raj held out. What they have in common: neither password is in the public list. `mariepw` is a pattern (username + `pw`), not a common password; `Gv7#pLm2!xQ` is long and random. The attack never reverses a hash: it only hashes guesses and compares, so a password that nobody guesses does not fall.

```
FINDING 1 — stored passwords (unsalted MD5)
  Cracked:          4 of 6 accounts, in 0.1 ms, with a public wordlist
  Why those fell:   123456, iloveyou, qwerty123 and password1 are in the public
                    list, and unsalted MD5 is fast: one cheap hash per guess,
                    compared against every digest at once
  Any that held out, & why: marie (mariepw: a pattern, not in the list) and
                    raj (Gv7#pLm2!xQ: long and random, not in any list)
  Severity:         High because a leaked table gives 4 of 6 real passwords in
                    under a millisecond with no special tool
  Fix (1 line):     store passwords with Argon2 and upgrade old ones on login
```

Remediation — migrate NotesApp to Argon2

The stubborn objection — "raj's password survived even MD5, so the real fix is to tell users to pick strong passwords; why add Argon2 at all?" Are they right? **(Our answer, decided together):** We disagree. raj's password was not cracked this time, but that only shows that this attack did not guess it; it does not show that MD5 is secure enough. A strong password makes guessing harder, and Argon2 makes each guess slower, so using both together is better. Also, we cannot guarantee that every user sets a strong enough password.

The three changes (as given in the lab).

1. `web/requirements.txt`: added `argon2-cffi==23.1.0`, then rebuilt the web image.

2. `web/app.py`: imports `PasswordHasher` and the Argon2 exceptions; one `ph = PasswordHasher()` just below `app = Flask(__name__)`; the helper `is_argon2_hash()` (true when the stored value starts with `$argon2`).
3. `login()`: the query now also selects `password` and no longer has `AND password = '...'` in the `WHERE` clause. The username part stays concatenated, as the lab asks (SQL injection is for December). The new block checks the password before `conn.close()`, and `if row:` became `if authenticated:`.

Our `except` clause names `VerificationError` and `InvalidHashError`. In argon2-cffi 23.1.0, `VerifyMismatchError` is a subclass of `VerificationError`, and `InvalidHash` is the same class as `InvalidHashError`, so the behaviour is the same as in the lab's code.

How the `else` branch works. If the stored value starts with `$argon2`, the account is already migrated: `ph.verify` checks the typed password against the hash. If not, the stored value is an old plain-text password. The app compares it once with the typed password. When they match, the login succeeds, `ph.hash` makes a new Argon2 hash (with a new random salt) of that password, and a parameterised `UPDATE` writes it back to the row. A wrong password never reaches the `UPDATE`, so it cannot change the stored value. From then on, that account uses the Argon2 branch.

Predict — before we look at the database (what does alice's stored password hold after one normal login, and why?): We think it will not still hold `alicepw`. If the lab upgrades the password storage after a successful login, the system turns the password into an Argon2 hash and saves that. Later logins verify the password against the hash, so there is no need to keep the original plain-text password.

Check it. Before: alice's stored password in the seed data (`db/init.sql`) is `alicepw`. After one normal login as `alice` / `alicepw` (12:01, a scripted HTTPS form login to `https://localhost/login`), the database shows:

```
$ docker compose exec db psql -U notesapp -d notesapp \
    -c "SELECT username, password FROM users WHERE username = 'alice';"
 username |                                                  password
----------+-------------------------------------------------------------------------------------------------------
 alice    | $argon2id$v=19$m=65536,t=3,p=4$g+I5JpMsPN94drAy9iW6Mg$y//nRWLVcPsinv8BtLBYVY/PgXFpt6ptwBfoiBM5Fd8
(1 row)
```

This is the lab's query. Our stack runs under the Compose project name `notesapp-lab`, so the real command also has `-p notesapp-lab` and our three `-f` Compose files.

Checkpoint — the app is in the right state. alice is migrated. We logged out and back in: the second login succeeded through the Argon2 branch, and the stored hash did not change. alice also logged in normally in Chrome at 13:40 (during the Lab 1 re-test); NotesApp logged `login success user=alice` with no new `upgraded` line, and the hash above is still the same. A wrong password was refused before and after the migration. The other 34 accounts are still plain text; each one upgrades at its own next successful login.

Finding 2 — How much did the fix actually buy? (measure)

Predict first (our guess before measuring — how many times slower per guess is Argon2 than MD5?): We guess that Argon2 may be 1,000 times slower than MD5, or even more, because it deliberately makes each computation cost more time and memory, so an attacker guesses passwords more slowly. The exact difference needs the speed test.

Measure. The lab's script, run once inside the `web` container (12:01; Python 3.11, argon2-cffi 23.1.0, default parameters `m=65536, t=3, p=4`):

```
MD5    : 3,459,762 guesses/second
Argon2 : 98 ms for ONE guess  (10.2 guesses/second)
```

Hash	Guesses / second	Guesses an attacker makes in one day (× 86,400)
MD5	3,459,762	298,923,436,800
Argon2	10.2	881,280

Ratio: 3,459,762 ÷ 10.2 ≈ 339,000. A 1,000,000-word list: 1,000,000 ÷ 3,459,762 ≈ 0.3 s with MD5; 1,000,000 ÷ 10.2 ≈ 98,000 s ≈ 27 h with Argon2, for each account.

```
FINDING 2 — the fix, measured
  Argon2 is about 339,000 times slower per guess than MD5.
  Verdict — with your stolen database, which hash does the
  attacker beat, which do they give up on, and WHY (one sentence):
  The attacker beats MD5 and gives up on Argon2, because a 1,000,000-word list
  takes about 0.3 s against MD5 but about 27 hours per account against Argon2.
```

Limits of this measurement: it is one run on one laptop CPU, not a GPU attacker. A password at the very top of a wordlist (for example `123456`) can still fall quickly: Argon2 makes each guess expensive, but it does not make a weak password strong.

Finished early (optional) — crack marie with a guessing rule

We added the rule `<username>pw` (6 extra candidates, 176 in total) and looped over `wordlist + rule_based`:

```
$ python3 crack/crack_rules.py
cracked 5 / 6 in 0.1 ms
  liwei : 123456
  aisha : password1
  tomk : qwerty123
  sofia : iloveyou
  marie : mariepw
```

marie now falls; only raj, the long random password, survives. Note for the audit: the wordlist size is not the limit; attacker creativity is. The only password that held was one with no pattern at all.